A DIGITAL IOT-BASED FOREST NETWORK

Final Year Project

Session 2022-2026

A project submitted in complete fulfillment of the requirements for the Degree of
BS in Computer Science



Authors

Areeba Shahid (UET/22-CS-34)

Nimra Faryal (UET/22-CS-36)

Afnan Fatima (UET/22-CS-59)

Supervised By

Dr. Farrukh Zeeshan Khan

Associate Professor

Department of Computer Science

University of Engineering and Technology Taxila

June 2026

Project Details

Project ID (for office use)	**2022-M-59**			
Type of project	✓	Traditional		Industrial
Nature of project	✓	Development		Research & Development
Sustainable Development Goals(SDGs)		Food Security		Sustainable Energy & Water
	✓	Climate Change and Environment		No Poverty and Zero Hunger
		Good Health and Well-Being		Quality Education
		Gender Equality	✓	Industry, Innovation, & Infrastructure
		Decent Work and Economic Growth		Life on Land & Water
Area of specialization	✓	Artificial Intelligence (AI)		Blockchain Technology
		Cybersecurity		Data Science and Analytics
		Game Development	✓	Internet of Things (IoT)
		Natural Language Processing (NLP)		Computer Vision & Image Processing
	✓	Web Development		Mobile App Development
		Robotics		Health Informatics

FYP Committee	
Dr. Syed Adnan Shah	**Dr. Qamas Gul Khan Safi**
Dr. Rao Wakeel Ahmad	**Mrs Asima Ismail**
Dr. Farooq Ali	

Acceptance Certificate

This is to certify that the Final Year Project titled **"A Digital IoT-Based Forest Network"** submitted by Areeba Shahid, Nimra Faryal, and Afnan Fatima, Registration Nos. 22-CS-34, 22-CS-36, and 22-CS-59, has been reviewed, examined, and accepted as satisfactorily fulfilling the academic requirements of the Bachelor's program in Computer Science at UET Taxila.

The work has been found complete, and presented in accordance with departmental standards & guidelines.

Name & Signature: Dr Farukh Zeeshan
Supervisor

Name & Signature: Dr Syed Aun Irtaza
Head of the Department

Abstract

Forest is a part of our living environment. They hold carbon, have diversity values and support global climate regulation. Today, unfortunately with global warming, man's presence and the high risk of fires in forests, many forests are under threat.By and large, however, forests are still monitored by occasional human visits or by low-tech, stand-alone local monitoring apparatus which risks missing important shifts in temperature, humidity or carbon-dioxide levels, especially in inaccessible areas. This project aims to build an IoT implementation monitoring system which utilizes two wireless sensor nodes placed within a forest area to carry out various measurements on key environmental parameters and hand them over to a gateway over a multi-hop 433 MHz radio-frequency link, which is implemented using a Raspberry Pi based micro-computer.

The gateway receives the data from the sensor and forwards them to the PostgreSQL relational database. This information is then exposed as a REST API via a flask web app and presented in a highly simplified (safety focussed) way for any General User, and then full technical details (including the actual values from the sensors, live risk alerts, risk alert management and administration of user accounts) for any Administrator. The system makes visual and audible notifications if temperature, humidity or CO_2 levels breach safe limits and the GPS location of each node, which is generated when it is active, is shown along with an Active/Inactive status.

As the finished prototype has shown, it is possible to assemble a modular, cheap and readily deployable forest-monitoring solution using ‘commodity' hardware and off-the-shelf software that doesn't rely on costly proprietary infrastructure or constant cellular connectivity, while using the current resources of local forest users. System validation was conducted via unit, integration and user acceptance testing which has demonstrated a robust, scalable system that can reliably transfer end-to-end data from the sensing nodes to the dashboard, and will serve as a solid basis to further the autonomous, large-scale deployment of forest sensing and monitoring systems in the future.

Author's Biographical Sketch

Areeba Shahid	Areeba Shahid is a final-year BS Computer Science undergraduate with interests in databases, backend engineering, and API design. For her Final Year Project she designed the relational PostgreSQL schema and implemented the Flask REST API, including the authentication layer and the administrative endpoints for alert and user management. She is particularly focused on building clean, well-structured backends that expose data reliably to client applications.
Nimra Faryal	Nimra Faryal is a final-year BS Computer Science undergraduate with a strong interest in embedded systems, the Internet of Things, and wireless communication. For her Final Year Project she led the hardware prototyping of the sensor nodes, designed and implemented the 433 MHz request-acknowledge-relay communication protocol, and developed the Raspberry Pi gateway ingestion software. Her interests lie in building affordable, real-world IoT solutions for environmental and safety-critical applications.
Afnan Fatima	Afnan Fatima is a final-year BS Computer Science undergraduate with interests in front-end development, human-computer interaction, and software testing. For her Final Year Project she built the role-based web dashboard, implemented the live-update and alerting logic, and led the system testing effort across the full pipeline. She is keen on designing clear, role-appropriate interfaces that make complex sensor data understandable to both technical and non-technical users.

Acknowledgement

We would like to express our sincere gratitude to our respected supervisor, Dr. Farrukh Zeeshan Khan, for his constant support, valuable suggestions, and detailed feedback throughout the development of this project. His guidance helped us understand the practical side of our work, refine our ideas, and stay motivated during every stage of the project.

We would also like to thank the Department of Computer Science, University of Engineering and Technology, Taxila, for providing the academic environment, laboratory access, and resources that enabled us to carry out our Final Year Project. The knowledge imparted by the faculty throughout our undergraduate program laid the foundation upon which this project was built.

We are grateful to the open-source community, including the developers of the RadioHead library, the Arduino and Raspberry Pi ecosystems, Flask, and PostgreSQL, whose documentation and freely available tools made this project possible. Finally, we extend our heartfelt appreciation to our families and peers for their unwavering patience, encouragement, and moral support during the long months of research, development, and writing.

Dedication

To our beloved parents, whose endless love, prayers, and unyielding sacrifices have been our greatest strength and inspiration throughout this journey. To our siblings and family members, whose constant encouragement kept our spirits high during challenging times. And to our respected teachers and mentors, whose invaluable guidance and wisdom have illuminated our path, shaping us into the professionals we aspire to be.

Declaration Certificate

We certify that the Final Year Project titled ″ **A Digital IoT-Based Forest Network** ″ submitted to the Department of Computer Science, UET Taxila is our original work, developed in accordance with academic standards and HEC guidelines. The plagiarism similarity index is **14 %**, within the acceptable limit of 20% (report attached as Appendix A). All external sources have been properly cited. We affirm the originality of this work and accept full responsibility for any future plagiarism claims.

Author Signatures:

Name: Areeba Shahid Signature: _____
Registration No.: 22-CS-34

Name: Nimra Faryal Signature: _____
Registration No.: 22-CS-36

Name: Afnan Fatima Signature: _____
Registration No.: 22-CS-59

Table of Contents

List of Abbreviations 14
Chapter 1. Introduction 15
1.1 Introduction 15
1.2 Problem Statement 18
1.3 Proposed Solution 19
1.4 Main Objectives 19
1.5 Assumptions & Constraints 20
1.5.1 Assumptions 20
1.5.2 Constraints 20
1.6 Project Scope 21
1.7 Software Development Lifecycle Model 21
Chapter 2. Existing Solutions 23
2.1 Selection Criteria for Solutions 23
2.2 Solution 1: Dryad Networks (Silvanet) 23
Overview 23
Key Features & Functions 23
Technology Stack 23
User Interface & Experience 23
Strengths 24
Drawbacks 24
Target Audience 24
2.3 Solution 2: Libelium Smart Environment 24
Overview 24
Key Features & Functions 24
Technology Stack 24
User Interface & Experience 24
Strengths 24
Drawbacks 24
Target Audience 25
2.4 Solution 3: Rainforest Connection (Guardian) 25
2.4.1 Overview 25
Key Features & Functions 25

Technology Stack 25
User Interface & Experience 25
Strengths 25
Drawbacks 25
Target Audience 25
2.5 Solution 4: Pano AI 25
Overview 26
Key Features & Functions 26
Technology Stack 26
User Interface & Experience 26
Strengths 26
Drawbacks 26
Target Audience 26
2.6 Solution 5: OroraTech 26
Overview 26
Key Features & Functions 26
Technology Stack 26
User Interface & Experience 27
Strengths 27
Drawbacks 27
Target Audience 27
2.7 Comparative Analysis Table 27
2.8 Identified Gaps & Project Justification 27
Chapter 3. Requirement Analysis 29
3.1 Stakeholders List 29
3.2 Requirements Elicitation 29
3.2.1 Functional Requirements 29
3.2.2 Non-functional Requirements 30
3.2.3 Requirements Traceability Matrix 30
3.3 Use Case Description 31
3.3.1 Use Case I: UC-01 — Monitor Environmental Parameters 31
3.3.2 Use Case II: UC-02 — View Real-Time Dashboard 31
3.3.3 Use Case III: UC-03 — Receive Alert Notifications 32
3.4 System Constraints and Assumptions 32
Chapter 4. System Design 33

4.1 Use Case Design 33
4.2 Activity Diagram 34
4.3 Sequence Diagram 35
4.4 Software Architecture Diagram 36
4.5 Class Diagram 37
4.6 Database Diagram 38
4.7 Collaboration Diagram 39
Chapter 5. Implementation 41
5.1 Work Breakdown Structure (WBS) 41
5.2 Team Roles and Responsibilities 42
5.3 Tools and Technologies 42
5.4 Implementation Details 42
5.4.1 Hardware Implementation 43
5.4.2 Communication Protocol 43
5.4.3 Gateway and Database 43
.4.4 Backend API 44
5.4.5 Web Dashboard 44
5.4.6 Alert Threshold Configuration 45
5.5 Screenshots of Prototype / System 45
5.6 Challenges During Implementation 47
Chapter 6. System Testing 49
6.1 Test Cases Design 49
6.2 Unit Testing 50
6.3 Integration Testing 50
6.4 User Acceptance Testing 51
Chapter 7. Conclusions 52
7.1 Summary 52
7.2 Recommendations for Future Work 52
Chapter 8. References 54

List of Tables

Table 1.1. Comparative Analysis of Existing Solutions 27
Table 3.1. Stakeholders List.....29
Table 3.2. Requirements Traceability Matrix 30
Table5.1. Work Breakdown Structure (WBS) 41
Table5.2. Project Schedule (Sep 2025 – Jun 2026) 41
Table 5.3. Team Roles and Responsibilities 42
Table 5.4. Tools and Technologies 42
Table 5.5. Alert Threshold Configuration.....45
Table 6.1. Test Cases Design and Results (*range-dependent) 49

List of Figures

Figure 1.1. Components of the Internet of Things (IoT).....16
Figure 1.2. Conceptual IoT-based Forest Monitoring Network.....18
Figure 2.1. Use Case Diagram for the Forest Monitoring System.....33
Figure 2.2. Activity Diagram for the Sensor Data Relay Cycle.....34
Figure 2.3. Sequence Diagram for End-to-End Data Flow.....35
Figure 2.4. Layered Software Architecture of the Forest Monitoring System.....37
Figure 2.5. Class Diagram for the Forest Monitoring System.....38
Figure 2.6. Database Schema (ERD) for the Forest Monitoring System.....39
Figure 2.7. Collaboration Diagram for Gateway-to-Dashboard Data Flow.....40
Figure 5.1. PostgreSQL Schema Execution in pgAdmin.....44
Figure 5.2. Forest Monitoring Dashboard — Live Node Readings.....46
Figure 5.3. Role-Based Login Screen.....46
Figure 5.4. Administrator — Manage Users Panel.....47

List of Abbreviations

Abbreviation	Full Form
IoT	Internet of Things
WSN	Wireless Sensor Network
RF	Radio Frequency
ASK	Amplitude Shift Keying
OOK	On-Off Keying
CO_2	Carbon Dioxide
ppm	Parts Per Million
API	Application Programming Interface
REST	Representational State Transfer
SQL	Structured Query Language
ERD	Entity-Relationship Diagram
UML	Unified Modeling Language
USB	Universal Serial Bus
GPS	Global Positioning System
LoRa	Long Range (radio)
ACK	Acknowledgement
SDLC	Software Development Lifecycle
FYP	Final Year Project

Chapter 1. Introduction

1.1 Introduction

Sustainability has become a key element in the environmental and natural resource management in the past few decades. Sustainability according to the World Commission on Environment and Development Report is "meeting the need of the people of the present without endangering the ability of the future generations of people to meet their needs" [1]. Here, forests are viewed as components of the ecological sustainability system, and play a very significant part in regulating climate, sequestering carbon, maintaining biodiversity and providing for lifelines and raw materials to millions of people. Marchi et al. (2018) proposed the new concept named Sustainable Forest Operations (SFO) on how the forest resources can be developed and management to balance the economic, social and environmental. In SFO forestry jobs are shifting to use data and the Scientific method to manage forests in a sustainable and responsible manner.

As the technology matures, forestry is gradually moving towards the digital and intelligent era of forestry management, where adoption of the Internet of Things (IoT) [3] and Wireless Sensor Networks (WSN) as well as the use of cloud computing and big data analytics [4] will play a significant role. IoT (Internet of Things) is where the network of devices and sensors can collect, process and communicate data without human participation. Within forestry, IoT could track environmental factors such as temperature, humidity, CO_2 levels and more in the forest. The sensor nodes utilize various communication methods such as RF modules, LoRa, GSM, Wi-Fi or satellite links and send the real-time data to centralized gateway and databases for visualization and further analysis [4]. This can be called 'digital forestry' and will help in making better decisions, early warning of ecological risks and the ability of forest departments and research institutes to remotely track forest health, use of forest resources, and changes in climatic parameters.

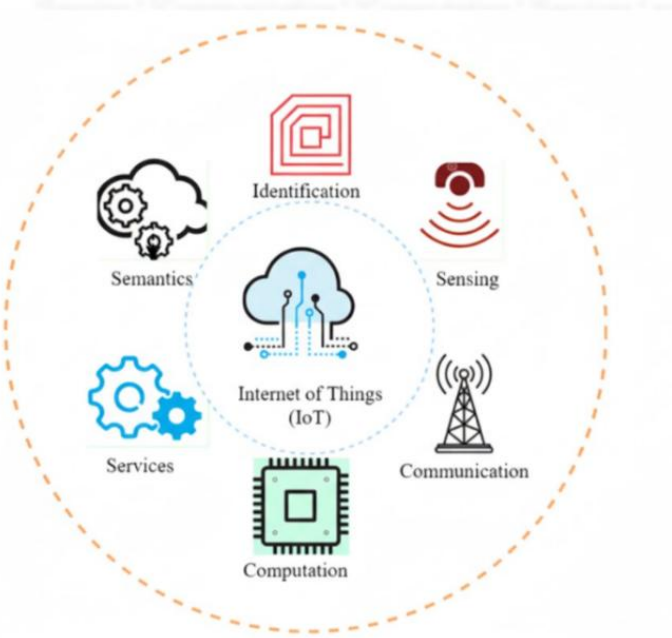


Figure 1.1. Components of the Internet of Things (IoT)

Figure 1.1 outlines the six fundamental pillars of an IoT system: Sensing gathers environmental data; According to Figure 1.1 there are six pillars of an IoT system: Sensing gathers environmental data; Communication transmits it; Computation processes it; Identification uniquely names devices; Services provide functional applications; Semantics is to make sure that the data is intelligently understood. All of these elements provide the capability of the forest-monitoring system developed in this project, including the role-based dashboard, the REST API and the gateway software, which are provided under the category of computation, services, and semantics, and the DHT11 and MQ135 sensors and radio communication, which are provided under the category of sensing and communication.

Despite the importance of forests, they continue to face severe challenges worldwide. As a result deforestation, loss of biodiversity and reduction in forest health is occurring through environmental degradation, climate change [6] and anthropogenic activities [7]. Global warming has led to extended periods of drought and irregularities during seasons that make forests more vulnerable to fires [8]. Destruction of forest cover by illegal logging and timber smuggling is causing a loss of forest cover, particularly in tropical regions, and forest disease, soil erosion, invasive species introduced into forests and decreased rainfall are further damaging forest ecosystems. Land that is burned just from forest fires alone annually kills thousands of hectares annually releasing tons of CO_2 and particulate matter into the air and causing imbalance in the climate and health risks. These challenges emphasize the importance of smart systems which can handle and sense any change in the environment in the early stage.

The project identifies two main and related issues to be tackled which are: Environmental degradation and Early fire-risk detection. The current forest monitoring methods are primarily manual with efforts to cover every forest, calling for time-consuming field visits, environmental inspections and inconsistent data coverage. Forest is very large and hard to reach, data collection by human teams may not be possible to do all the time. This means that key fluctuations in temperature, humidity or unexpected CO_2 levels typically are only discovered when they reach extreme levels that create fire or ecological messes. Moreover, the scattered and outdated data storage and the absence of automation means that authorities do not have timely information for necessary decision making. It is essential, therefore, that environmental surveillance moves from reactive monitoring to real-time, automated, and integrated monitoring, for efficient forest management.

To observe forests and identify fires, many technological solutions are proposed such as satellite imaging, drone, thermal cameras and meteorological models. Although Satellite images offer a large scale forest coverage information, there are disadvantages such as low temporal resolution, chances of cloud obstruction, delay in data availability [12]. Though drones can take higher granularity images, they need human intervention and cannot run for long periods of time because of power constraint [13]. Smoke or fire can be detected by camera-based systems or watchtowers, however after the visible fire has occurred and a delayed response as a consequence. Several IoT based wildfire-detection systems and AI based prediction models have been proposed in recent research works but many of them are based on a single factor like temperature, smoke etc. and provide no integrated, connected and real-time monitoring platform [13]. Existing solutions are consequently incomplete and not suitable for performing environmental monitoring.

There is a lack of systems currently on the market that are scalable, easy to maintain, inexpensive, and integrate data for monitoring forests at a significant cost and/or maintenance burden. The vast majority of the state-of-the-art high-tech solutions, whether they be satellite-based fire detection or drone fleets, are costly, require a high degree of skill for operators and are mostly imported, not to mention being hard to adopt in developing areas [11]. Many systems currently stand alone on their own with no integration of data into a centralised database, resulting in data fragmentation and making it difficult to carry out long term trend analysis. Further, there are network connectivity issues in dense forests and there is no steady supply of electric energy for uninterrupted monitoring of electronic systems [15]. Real time, automated alert systems are not available resulting in authorities only reacting after damage occurs. Thus, a system is a much-needed requirement that is both inexpensive and locally deployable, with the capability to provide on-going data collection, central storage, and early-warning notifications.

In order to fill these gaps, this project adopts Wireless Sensor Network (WSN) and IoT (Internet of Things) for the implementation of the Forest Monitoring System. The system included two sensor nodes for

continuous monitoring of temperature, humidity and CO_2 concentration[16]. The first node will use 433MHz RF communication to send it's reading to the second node which will acknowledge the received reading and add its own reading and then send it to a bridge microcontroller that's connected to the Raspberry Pi gateway. All information is passed to and retained in a relational database that is understood and fed by the gateway, which makes it accessible via a web dashboard with role-based access. Users are alerted immediately when dangerous conditions are detected, by means of a flashing visual banner and a sound alarm system.

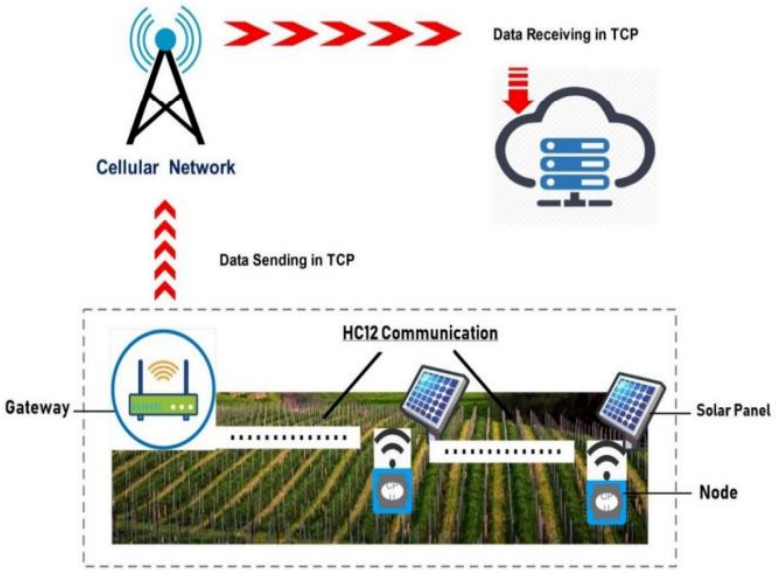


Figure 1.2. Conceptual IoT-based Forest Monitoring Network

Sensors (nodes) are deployed throughout the forest to collect environmental parameters as illustrated in Fig. 1.2. Every node has the microcontroller, which is linked to the sensors it is equipped with as well as the RF for short range communication. It passes the data from node to node until it arrives into gateway node which sends the data to database server where it can be stored and retrieved. The conceptual diagram provides a view of the solar-powered, cellular-connected deployment as the final goal, but the current prototype de-emphasizes the use of solar power and the local network to validate the core sensing, multi-hop relay, ingestion and dashboard pipeline using mains/USB-powered deployments.

1.2 Problem Statement

Lastly, to allow forests to stay in a good state, it's necessary to always keep track of the temperatures, humidity and the carbon dioxide concentration. Currently, those factors are being manually monitored or monitored by individual sensors sluggish at reporting or inconsistent, offering a weak control on monitoring what's going on from far off. Satellites and drones cover large areas with high cost, but lack ground-level continuous monitoring. Most of the low cost IoT solutions consist of limited number of parameters or do

not offer remote role-based access to the data collected. The gap, thus arises for an inexpensive, real-time, multi-parameter measuring system which stores data at a central location and securely displays the data to various user classes.

This monitoring gap has real-world implications. Forest fires often start off as small, compact bursts of fire, patterns that have been detected early on the ground by their ground-level temperature rise, rapid dehydration of the air, and CO buildup, but hardly by satellite or air-based systems during the most critical first several minutes of a fire. Fires large enough to be detected by satellite or a drone may be structurally burned out, or may have passed the point where it can be most easily and cheaply suppressed. Collecting these variables continuously at ground level and alerting in real-time in an alarming manner therefore fills a major gap or blind spot that exists in the current practice such as for the forest authorities in developing areas where they cannot afford imported satellite and drone solutions.

1.3 Proposed Solution

Forest Monitoring and Early fire-risk Warning is a challenging task which can be overcome by proposing a smart, real-time forest-monitoring system by using Wireless Sensor Network (WSN) and IoT technologies. Both sensor nodes continuously sense their environmental parameters (Temperatures, Humidity and CO_2 level) and send these values wirelessly according to a request-acknowledge-relay protocol over 433 MHz RF, via a dedicated bridge microcontroller and USB serial link to a Raspberry Pi Gateway.

The proposed system is better than manual and/or isolated forest-monitoring methods and it is also low in cost and it is operational in real time round the clock as compared to satellite or drone (UAS) based system. The system allows authenticated users to access a secure Web interface, while automatically notifying them when conditions surrounding the environment or fire-hazard become abnormal. The project's likely outcome will be rapid detection of ecological anomalies, and the creation of a central and dependable data repository – that is, a dependable decision support system for forest-resource management. It is modular and extensible, new nodes can be added to it without significant changes to the infrastructure in other modules.

1.4 Main Objectives

- To construct a system to take readings of the temperature, humidity, CO_2 concentration of the forest and keep it real-time.

- To create a wireless sensor network based on IoT to collect environmental information automatically.
- To establish a centralized database and web-based dashboard for delivering real-time environmental visibility.
- To create and apply an automatic triggering mechanism, such as an audible warning, that is triggered when instrumentation readings exceed safe levels, which can be used to help detect an early fire threat.
- To enable visualization of historical data for trend analysis and forest-management decision.
- To have a modular system that can be extended to add more nodes with minimum infrastructure changes.
- To implement secure, role based access for administration / user and ensure registration process is approval gated.

1.5 Assumptions & Constraints

1.5.1 Assumptions

- The chosen forest / test region has a very basic but sufficient wireless connectivity between nodes and between nodes and gateways.
- Providing enough and perpetual electricity to the sensor nodes during running.
- Internet-enabled devices like Smartphones or Laptop are provided to the authorized users including Forest officials, researchers and administrators.
- Environmental values are within typical ranges of the chosen sensors.
- All the sensing and communication components operate well and there are no significant hardware failure during the testing period.

1.5.2 Constraints

- **Low Range:** ASK/OOK 433 MHz RF modules are low cost and have limited range, and they need external wire antennas to get a reliable reception.
- **Power Constraint:** MQ135 consumes a lot of current on its heating element and can be a problem if the Arduino is powered just by USB port.
- **Hardware Availability**: The number of available board of Arduino and pairs of RF-modules provided a limit to testing.
- **Network Constraint**: For the time being, both the gateway and database server have to be on the same local network.

- **Geographic Scope:** This prototype was tried only over a limited range and distance and not over an entire forest reserve.

1.6 Project Scope

This project includes design; wiring; firmware for two sensor nodes and one bridge node, software for the Raspberry Pi gateway; relational database schema, and the Rest API; role based Web Dashboard and Alerting mechanism. The scope has intentionally been limited to the design of the enclosure for outdoor use in a short-term timeframe, as well as the management of the solar/battery system, which are seen as future research directions, and integration with the national disaster-management authorities. The project is implemented as a functioning prototype, instead of a total field deployment, to ensure the end-to-end functioning of the complete sensing-to-dashboard pipeline within the time and hardware constraints of the project, thus proving the core technical contributions.

1.7 Software Development Lifecycle Model

The project was a bottom up, iterative development philosophy, whereby each sub system was tested and validated separately and sequentially. This is because the project involved both hardware and software, and because it was easier to test each layer individually (sensing, communicating, ingesting, storing, serving and presenting) so that it was easier to identify and resolve integration issues. The main activities were:

- **Requirement Analysis:** Identifying Sensors, Range of communication and requirements of dashboard.
- **Hardware Prototyping:** wire-up and test each sensor node one-by-one.
- **Communication Layer:** implement and test the RF send/acknowledge/relay protocol.
- **Gateway Development:** Construct a serial to database ingestion pipeline.
- **Backend & API:** define the API/Database structure.
- **Frontend Development:** implementation of live updating and alerts in the dashboard that will be role specific.
- **Testing & Refinement:** verify the flow of the data end to end and resolve integration problems.

The purely sequential (or waterfall) model was explored but was found to be unsuitable for this project as it requires the requirements and technical risks to be well understood in advance of implementation. Building required to discover some of the most important requirements such as the realistic reliable range of the RF modules and the power behaviour of the gas sensor in a combination of unfamiliar embedded hardware, unproven low cost radio link and multitier software. This accommodated in the iterative

incremental model, where each iteration through the activities above resulted in a working, hence implementable, increment of the system and the things learned (such as "we need to have a single shared radio driver") were fed back to the requirements and design of the next increment. This feedback loop is what finally resulted in a reliable, end-to-end pipeline, as opposed to one fixed specification up front.

Chapter 2. Existing Solutions

Objective: Existing solutions have been analysed and indicate gaps in costs, real-time space and integration, motivating the design choices in this project. Five popular commercial and non-profit solutions were studied extensively and compared with that required for a low-cost, locally deployable forest- or wildfire-monitoring system.

2.1 Selection Criteria for Solutions

The criteria to select the comparable solutions were the following. Market relevance has been taken into account to determine the technologies that are well known and trusted in the market. The focus was creating an IoT interface that is used for live monitoring and access remote. It was safety and responsiveness to fires and the environment that were given a first priority. Reliability and usability in use were evaluated from user feedback, and scalability and efficiency of the underlying sensor technology including accuracy, connectivity and cloud capabilities, were assessed. These criteria, combined together, provide a measure to ensure that the comparison is against truly competitive and representative solutions and not niche or obsolete products.

2.2 Solution 1: Dryad Networks (Silvanet)

Overview

Located in Germany, Dryad Networks is a company providing a ultra-early wildfire detection service called Silvanet based on a forest-wide sensor mesh that is directly deployed on the site using a wide-area LoRaWAN. One of the more prominent commercial systems for fire detection particularly during the smouldering stage, prior to the onset of open flame.

Key Features & Functions

Automated alerts, AI-driven edge processing, cloud based Dashboard and LoRa MESH network. The continuous detection principle of the sensors allows them to “sniff” the air for gases emitted in the early smouldering phases of a fire.

Technology Stack

Map and alerting dashboard displayed on the Web with information about the location and status of sensors and fire alerts within the area of deployment.

User Interface & Experience

A web-based map and alerting dashboard that shows sensor locations, status, and fire alerts across the deployment area.

Strengths

Very early warning systems - detects fire in minutes with the capacity to sniff for gases, the nodes are solar powered and maintenance free and are intended for long-life but outdoor use.

Drawbacks

Expensive to deploy on a wide scale, proprietary and closed-sourced and cannot be customized; no other option for adopters except recurring subscription model.

Target Audience

Large-scale forest owners and private owners, governments, and electric and gas utilities looking for early detection of wildfire in public or private forests.

2.3 Solution 2: Libelium Smart Environment

Overview

Libelium is a Spanish company that creates general purpose environmentalmonitoring IoT hardware solutions. It offers flexible multi-sensors nodes in its Smart Environment product line which can be programmed for a plethora of monitoring use-cases, including forests.

Key Features & Functions

Multi-sensor nodes with the ability to measure a variety of gases, CO_2, temperature, and humidity, and configure sampling/transmission.

Technology Stack

Waspmote sensor boards, Meshlium gateway and integration with third party platforms: cloud.

User Interface & Experience

Custom dashboards atop the data streams – orientated towards integrators, not end users.

Strengths

High versatility in sensing range and a well developed ecosystem of hardware that can be adopted to numerous environmental uses.

Drawbacks

Limited built-in fire specific alerting; hardware relatively expensive and normally needs integration effort to achieve a turnkey forest-monitoring solution.

Target Audience

Institutions and environmental organisations creating custom monitoring deployments, smart-city integrators.

2.4 Solution 3: Rainforest Connection (Guardian)

2.4.1 Overview

Rainforest Connection is a non-profit environmental group, using Guardian devices to turn recycled smartphones into solar-powered acoustic sensors monitoring rainforests for the noise of illegal logging and other dangers.

Key Features & Functions

Real-time acoustic threat analysis, AI-powered sound recognition for chainsaws, vehicles and other unusual sounds.

Technology Stack

AI sound-recognition models, a cloud back-end, and devices powered by solar cells that collect energy to drink from recycled phones.

User Interface & Experience

A mobile app with notification to rangers & partners on location.

Strengths

Very powerful in detecting illegal logging and human threats with a good sustainability and conservation aspect.

Drawbacks

Audio based detection does not directly detect fire parameters and is not as effective if used for early smoke or fire detection; it is a different threat model than environmental fire-risk monitoring.

Target Audience

Rainforest protection groups and conservation organizations with anti-poaching and anti-logging emphasis.

2.5 Solution 4: Pano AI

Overview

Pano AI offers a wildfire-detection service using camera, which can be installed on tower and uses AI smoke recognition to get fast visual confirmation of fires within large areas.

Key Features & Functions

Panoramic camera footage, smoke detection using AI and satellite data for crosschecking.

Technology Stack

HD panoramic cameras, artificial intelligence vision models, satellite integration and cloud intelligence dashboard.

User Interface & Experience

An operations dashboard that enables monitoring centers to successfully identify and triage detected events visually.

Strengths

Widespread coverage and quick visual alarm, minimising false alarms and aiding to prompt and quick human decision-making.

Drawbacks

It can only be installed if clear line of sight is available to be used and visibility conditions have a significant impact on detection; pre-flame gas signatures are not possible; has high installation costs.

Target Audience

Large landscape covered by utilities, fire services, and governments.

2.6 Solution 5: OroraTech

Overview

OroraTech is a German Company that supplies wildfire surveillance data based on images transferred over thermal satellite systems, tracking heat hotspots around the world and providing a Cloud Platform for Fire Intelligence.

Key Features & Functions

Thermal satellite imaging, global hotspot tracking and real-time wildfire maps.

Technology Stack

Thermal-imaging satellites (including specific nanosatellites) and a cloud platform for analytics.

User Interface & Experience

Availability of real-time wildforest fire maps and warning via a web platform.

Strengths

Using a coverage that is really big, worldwide, without requiring any ground infrastructure.

Drawbacks

Lower spatial resolution than ground sensors, and inherent orbital and cloud-processing delays mean detection is not as immediate or local as in-forest sensing.

Target Audience

Global agencies, governments and insurers requiring global fire intelligence.

2.7 Comparative Analysis Table

Table 2.1 summarizes the five reviewed solutions alongside the system proposed in this project, comparing their key features, underlying technology, pricing model, and principal strengths and weaknesses.

Table 1.1. Comparative Analysis of Existing Solutions

Product	Key Features	Technology	Pricing	Pros	Cons
Dryad Silvanet	Gas sensors, ultra-early detection	LoRa, solar, AI	Subscription	Minutes-level detection	Expensive at scale
Libelium	Multi-parameter sensors	Waspmote, Meshlium	Hardware sale	Versatile sensing	Limited fire alerts
Rainforest Guardian	Acoustic threat detection	Solar phones, AI audio	Nonprofit	Illegal-activity focus	Not fire-specific
Pano AI	Camera-based smoke detection	HD cameras, AI	Enterprise	Visual confirmation	Line-of-sight needed
OroraTech	Satellite thermal monitoring	Satellites, cloud	Subscription	Global coverage	Resolution / delays
Proposed System	Multi-node RF relay, role-based dashboard, live audible alerts	Arduino, 433 MHz RF, Raspberry Pi, PostgreSQL, Flask	One-time, low-cost	Customizable, open-source	Limited RF range (prototype)

2.8 Identified Gaps & Project Justification

Current deployed commercial wildfire detection systems such as camera towers, satellite systems, and proprietary LoRa networks are expensive to deploy, weather dependent, and rely on only one modality of detection. All the systems examined lack the attributes of being low-cost, multifunctional sensors in the forest, customizable via open source, and accessible locally on a role basis. In meeting the above, this project will develop an economical, IoT-based, multi-parameter sensor node (including temperature, humidity and CO_2) to provide 'early warning' of fire; an inexpensive 433MHz relay mechanism to allow for large-scale coverage; and a fully customisable, self-hosted web dashboard with real-time, role-based alerting to overcome the subscription fees and closed-source limitations of the commercial solutions reviewed, while ensuring the system is feasible to deploy in a developing-region context with limited resources.

Chapter 3. Requirement Analysis

The stakeholders of the system, functional, and non-functional requirements are identified and documented in this chapter, along with the principal use cases, which are described in detail along with the constraints and assumptions that limit the scope of the prototype

3.1 Stakeholders List

Table 3.1 provides a summary of the stakeholders of the system, their roles and their interest and/or influence in the project. It was crucial to involve these stakeholders early on so that the requirements reflect both the operational requirements for the forest authorities and the technical and academic requirements for the development and supervision team.

Table 3.1. Stakeholders List

Stakeholder	Role	Interest / Influence
Forest Authorities / Rangers	Primary users	Monitor conditions and receive early-warning alerts
Environmental Agencies	Secondary users	Use historical data for analysis and reporting
Project Supervisor	Academic supervisor	Guidance, evaluation, and approval
Development Team (Students)	Developers	System design, implementation, and testing
System Administrator	Technical manager	Maintain server, database, and data security
Local Government / NGOs	External stakeholders	Policy support and environmental planning

3.2 Requirements Elicitation

The team used document analysis on previous works in the field of environment monitoring using IoT and brainstormed the existing problems related to different approaches of forest monitoring and performed direct observation to get insights regarding the use of practical hardware (e.g. RF range and sensors' power consumption) while prototyping. These measurements were pragmatic and fed straight into the requirements both for the functional requirements and the non-functional requirements, ensuring that the documented requirements were within what the hardware could deliver.

3.2.1 Functional Requirements

- Each sensor node shall measure temperature, humidity and CO_2 level.
- All sensor nodes shall use 433 MHz RF request-acknowledge-relay communications.
- The gateway to parse the serial data and update it into a PostgreSQL database.
- The system shall have REST API for giving the latest readings, historical readings, and alerts.

- There shall be secure login as Administrator/User.
- New users shall be able to register themselves on the system (Conditional to Administrator approval).
- Administrators shall be able to add, approve and delete user accounts (Administrator accounts will be protected from deletion).
- An automatic alert shall be provided if any of the sensors (temp, humidity or CO_2) detects levels outside of predetermined targets.
- The system shall produce a sound alarm message to notify the operator of a newly detected high risk condition.
- The systems shall show the Active/Inactive status of each node, dependent on the data they last received and show their GPS co-ordinates.

3.2.2 Non-functional Requirements

- **Performance:** sensor readings in normal radio conditions should be transmitted and displayed on the dashboard almost instantaneously.
- **Reliability:** System should reliably communicate sensor information, minimising data dropouts in normal radio conditions.
- **Scalability**: the introduction of new nodes should not be expected to necessitate major changes in the architecture.
- **Security**: two level password protection should be employed - at client and API layer, and the Administrator user cannot be deleted.
- **Usability**: dashboard must provide a clear and role-specific view which is visually different to the technical audience and the non-technical audience.
- **Maintainability**: use a modular architecture for the firmware and software to enable the independent and successive upgrade of each layer.

3.2.3 Requirements Traceability Matrix

The Requirements Traceability Matrix (Table 3.2) links each key requirement to its source and to the method by which it was verified, ensuring that every requirement was accounted for during testing.

Table 3.2. Requirements Traceability Matrix

Req. ID	Requirement	Source	Verified By
FR-01	Sensor data acquisition	Stakeholder need	Hardware testing
FR-02	Node-to-node RF relay with ACK	Stakeholder need	Serial-monitor logs
FR-03	Gateway-to-database ingestion	Functional design	Database-query testing

Req. ID	Requirement	Source	Verified By
FR-05	REST API for live/historical data	Functional design	API-endpoint testing
FR-06	Role-based login & approval	Security requirement	Manual UI testing
FR-09	Threshold-based alerting with sound	Stakeholder need	Manual UI testing
NFR-01	Low dashboard-refresh latency	Performance need	Manual observation

3.3 Use Case Description

3.3.1 Use Case I: UC-01 — Monitor Environmental Parameters

Actor: Sensor Node.

Description: This use case demonstrates how the distributed sensor nodes collect environmental data in an automatic way in the forest. Every node automatically and continuously takes measurements of temperature, humidity and CO_2 without an operator.

Requirements: Sensor node must be powered, DHT11 and MQ135 sensors must be properly connected to the Arduino, and the RF transmitter/receiver modules must be working properly.
Main Flow: (1) the node sends an RF wireless signal to the next node or to the gateway periodically, based on the node's temperature, humidity and CO_2 sensor readings; (2) the microcontroller formats the sensor readings into a packet.

Post conditions: successful delivery of latest environmental measurements to the gateway and storage for further processing.

3.3.2 Use Case II: UC-02 — View Real-Time Dashboard

Actor: Forest Official / Researcher (User) or Administrator.

Description: The person/machine access sees real time environmental information on the web dashboard providing a snapshot of forest condition through monitored nodes.

Preconditions: user has a valid and approved account; the dashboard can have network access to the backend API.

Main Flow: First the user logs in with his user credentials, the system authenticates the user and checks his role; after this, the Dashboard continuously gets the latest readings from the REST API and then renders

a view adapted to the user role: The Administrator gets a detailed, full numeric view plus raw tables, the User gets a view that summarizes safety-related data.

Post conditions: User can observe the current environmental conditions, historical trends, and node status for monitoring and decision making.

3.3.3 Use Case III: UC-03 — Receive Alert Notifications

Actor: Forest Official / Administrator.

Description: this use case explains the alerting mechanism that identifies abnormal environmental conditions high temperature, low humidity, or elevated CO_2 that may indicate a fire risk.

Preconditions: The following conditions are necessary: threshold values are entered into the system; sensor data is flowing in continuously; the dashboard is opened in the browser of the user.

Main Flow: 1) Once every interval, the incoming sensor reading is compared to specified thresholds 2) If it exceeds any of the thresholds, a new Alert record is added and a flashing alert indicator banner is shown 3) A message to alert is played in the browser, to capture people's attention.

Post conditions: Administrator/forest official is notified and is able to take preventive action.

3.4 System Constraints and Assumptions

Constraints: the prototype's scope and performance is limited by a number of technical and operational constraints. Very little wireless communications is needed between nodes and gateway. Without stable power, sensor-nodes are not available and fault tolerance is not present. To access the dashboard, users need to have an internet connected device followed by an authorization. All the sensor behavior is assumed to be deterministic in normal environments and hardware integrity of all the sensing and communication devices is assumed during the testing process.

Assumptions: The test environment selected offers required base wireless connectivity to enable node to gateway communication. Throughout testing, the sensor nodes have reliable power. Users have device(s) with internet access like mobile phone or laptop and are authorized users. The energy consumption in environmental readings is within the range of operation of the selected sensors in typical scenarios and all energy sensing and communication components reliably run without causing significant failure of the hardware during the test period.

Chapter 4. System Design

This chapter shows the design of the system using the different types of UMLs and architecture diagrams which are directly drawn from the implemented Forest Monitoring System. The diagrams range from a top level depiction of how the actors interact to the dynamic behaviour of a single reading cycle to the static structure of the data model and the layered software design.

4.1 Use Case Design

The main relationships between the actors of the system: User, Administrator, and Sensor Nodes/Gateway (a system actor) are represented in the Use Case diagram shown in Figure 4.1. The User is able to see real time readings, alerts and node status and location. The Administrator may also add, approve, or delete users and clear alerts, and view raw database tables. The data is automatically supplied to the system by the Sensor Nodes and Gateway.

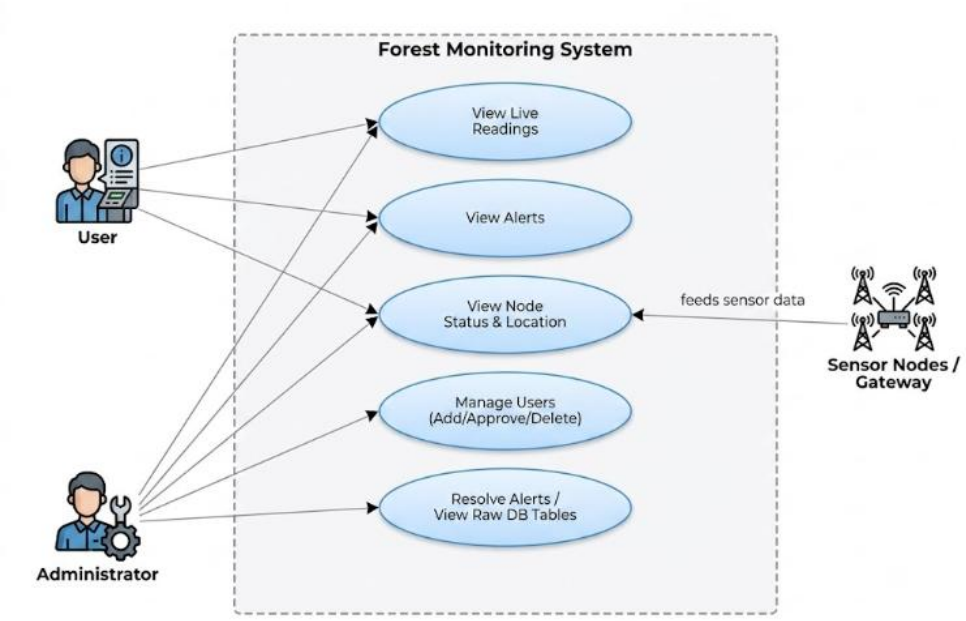


Figure 2.1. Use Case Diagram for the Forest Monitoring System

The separation of the two human actors at the use case level was an intentional design and not an after thought. This way the principle of least privilege is also directly included in the requirements via the distinct actors of User and the Administrator, each having their own subset of use cases and only the User having access to all of the common use cases. Interactions with the Sensor Nodes/Gateway can happen independently of any human request, hence it is considered as a non-human (system) actor. This framing

dictated a natural flow of thought as to what interactions needed to be authenticated versus what needed to be authorized using a role, versus what could be automated behind the scenes with no user interaction.

4.2 Activity Diagram

The Activity diagram (Figure 4.2) shows the sequence of operations for a single cycle to read the readings of the sensor. Data is read at node 1 and sent to node 2 which sends an ACK and reads identical data from its sensors, before sending a new packet to the bridge node, which sends it back to the gateway script via its USB serial port, where processing occurs, a DB update, etc., and finally, back to the dashboard for evaluation against alert levels. The decision node is titled in such a way to show that a specific branch is where the threshold breach opens an alert and plays an audible alarm.

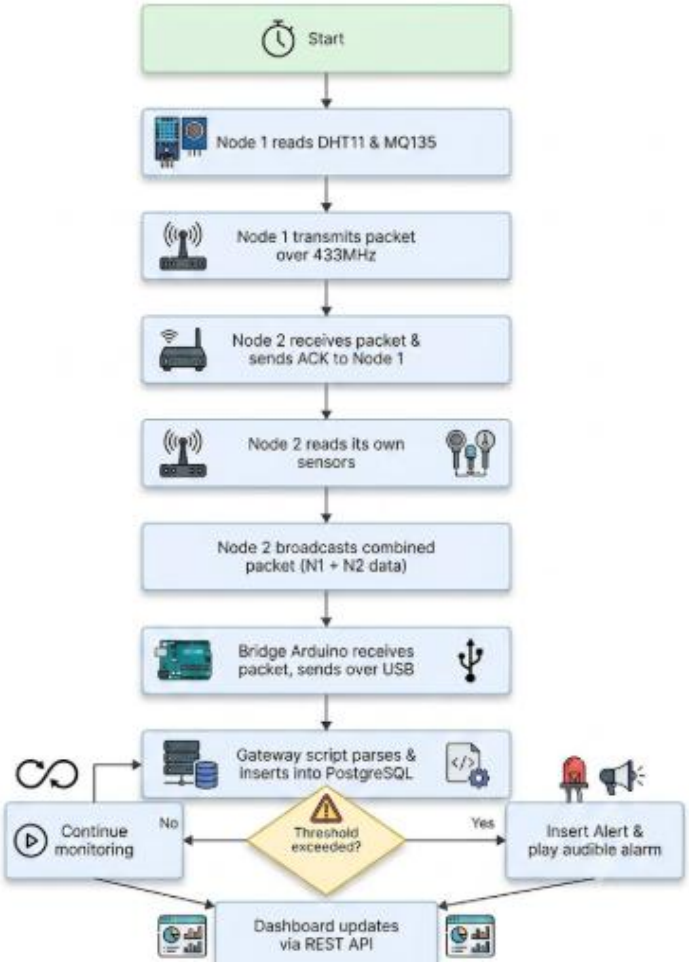


Figure 2.2. Activity Diagram for the Sensor Data Relay Cycle

The activity diagram was crucial to make explicit the handshake for requesting an acknowledgment of relay: it is a proving device which distinguishes this design from a "naive" broadcast. The transmissions

are NOT independent: Node 1 sends first, Node 2 waits till it has confirmed it got the message then adds its own contents and sends. The implementation of it as a separate act of "acknowledge" (as opposed to guaranteed delivery) revealed failure modes which had to be dealt with in the firmware: a lost packet results in Node 2 waiting for that same packet in the subsequent interval, and no corruption of the data stream. Making it so only one decision node at the end of a flow, where a reading is compared to its threshold, can create an alert, meant the behaviour was easy to test and it kept duplicate or conflicting alerts out of the way at varying stages in the flow.

4.3 Sequence Diagram

In the Sequence diagram (Figure 4.3) the interactions between the different nodes (Node 1, Node 2, the bridge communication Arduino, the gateway script, the database, and the web dashboard) are emphasized in terms of the time sequence. It depicts the RF communications and ACK, the USB serial communications to the Raspberry pi, the database insertion, and the periodic polling by the dashboard of this API to render new readings and set off alerts.

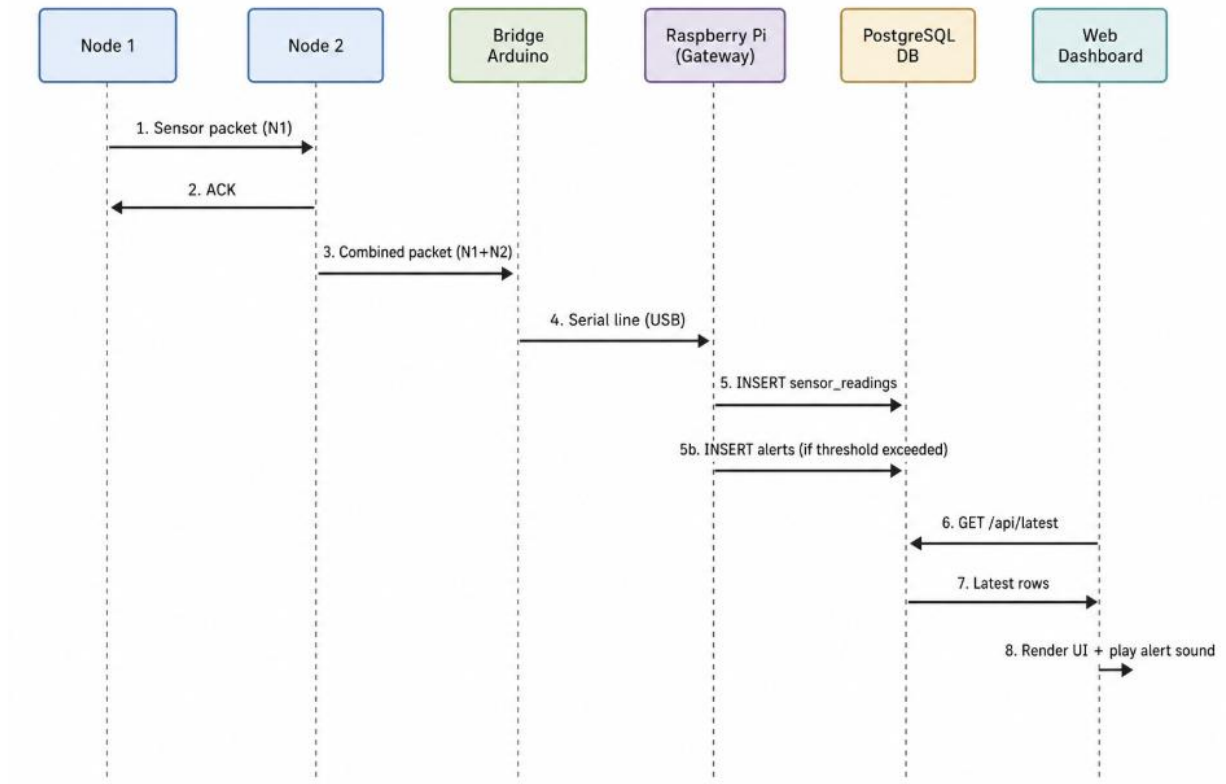


Figure 2.3. Sequence Diagram for End-to-End Data Flow

The activity diagram specifies what is done while the sequence diagram locks the dialog and the sender and receiver of each message, and which was extremely important in identifying timing faults during

integration. This makes clear that the Raspberry Pi gateway is passive on the serial line - that is, it waits on the lines for data from the bridge Arduino and never initiates a transfer, while the dashboard is active on the serial line and actively polls the REST API for new readings on a fixed period. This provides the freedom to manage the timescales of collection from the sensors to the database, and thereafter of delivering the data back to the dashboard on demand through the API. This means that a slow browser or a temporarily disconnected browser can't hold up ingestion and an influx of sensor messages can't overwhelm the front end and that each goes at their own pace against the same database.

4.4 Software Architecture Diagram

The Communication Layer comprised of the 433MHz RF link and the Arbux Bridge [USB serial connection of the bridge Arduino to the Raspberry Pi. Data is parsed and stored on its arrival by the Gateway/Ingestion Layer (gateway_insert.py). The Data Layer - PostgreSQL database. The Service Layer is based on a Flask REST API. The Presentation Layer is the web dashboard that is role based. Each layer is interfaces only with its adjacent neighbor layers ensuring loose coupling and easy testing of layers.

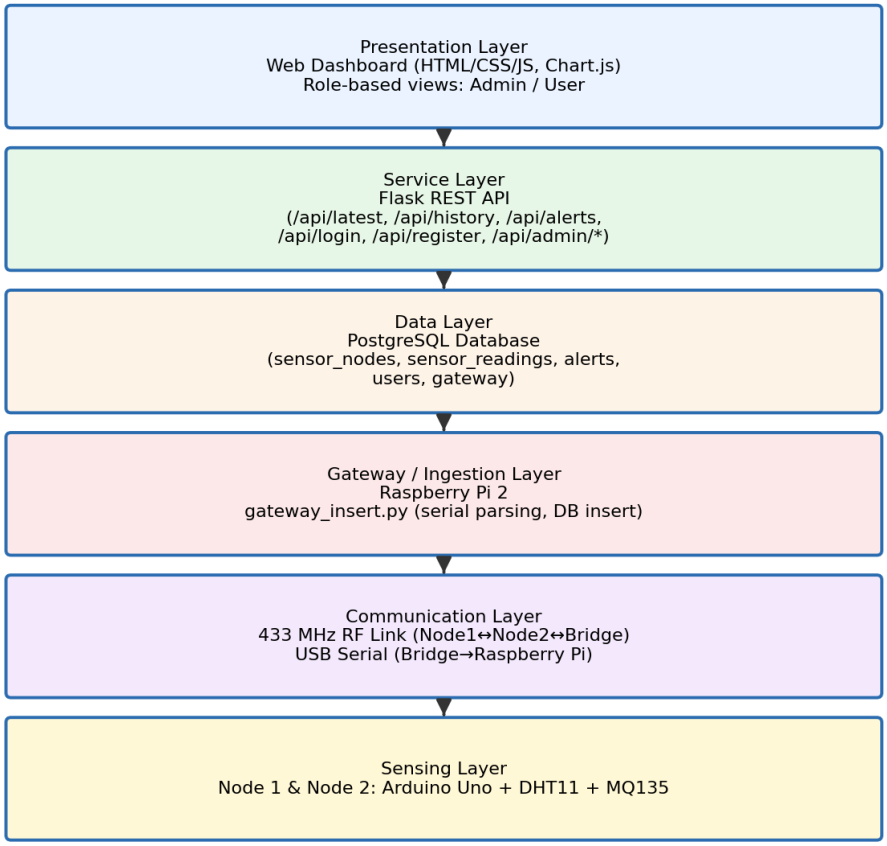


Figure 2.4. Layered Software Architecture of the Forest Monitoring System

The layered architecture is selected over a monolithic one partially in order to enable each of the different components of the system (embedded C++ firmware, ingestion script in Python, relational database, Python web service, front end in Javascript) to evolve separately and to be tested individually. Any change within a layer is encapsulated, since layer k only talks line-formatted serial to layer (k-1), which only talks SQL to the database, which only talks ON/Accelerated JSON over HTTP to service layer, which only talks JSON to the dashboard. The 433 MHz RF link, for instance, in the Communication Layer, could be swapped for a longer range LoRa module and the user wouldn't need to change a single line of code on the dashboard since the data contract crossing into the Gateway Layer – the line of comma-separated readings – would still be the same. This limitation of change is the one practical advantage that the layered model provided the project and one that allowed for incremental layer-by-layer testing.

4.5 Class Diagram

The Class diagram (Figure 4.5) presents a description of the main entities, which are the SensorNode, SensorReading, Alert, the User and the Gateway. There's one-to-one relationship between Gateway and SensorNode, as well as between SensorReading and one or many SensorNodes, and between Alert and one or many SensorReading. User is standalone with it's own instance of the authentication/approval behaviour.

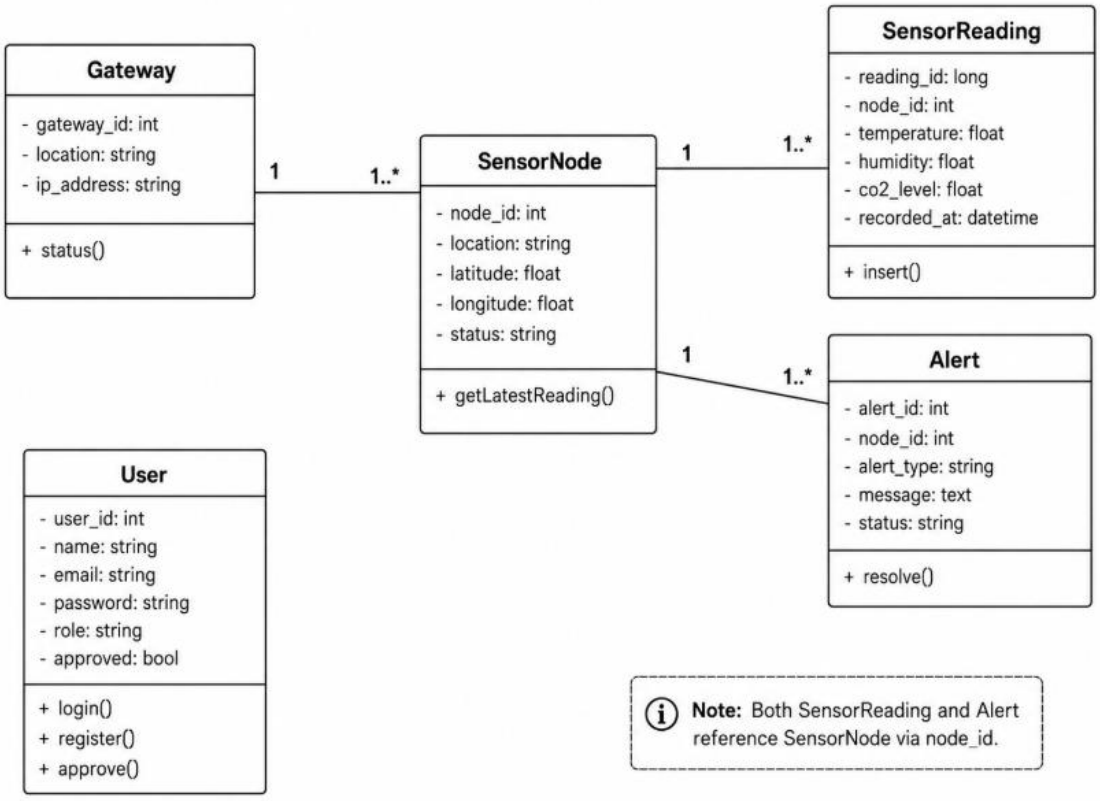


Figure 2.5. Class Diagram for the Forest Monitoring System

The class diagram avoids the creation of a dependency between the User entity and the sensing hierarchy. It might seem natural to link monitors to the nodes being monitored but this approach would have made linking accounts to their nodes subject to the physical deployment and made altering the set of nodes deployed more difficult. The separation between User and other entities results in an account being added, approved or removed from the model with no impact to the sensing model and the very opposite occurring when a node is added. The one-to-many relationships from SensorNode to SensorReading and to Alert reflect the real world, and the fact that a SensorNode raises a continuous stream of SensorReading objects, and, at times, a small number of Alert objects — making Alert a first class object rather than just a flag on a reading means that an alert can have its own life cycle, separate from the reading that triggered it: Resolved, Raised, Viewed.

4.6 Database Diagram

The alerts table also contains references to sensor_nodes. The sensor_nodes table optionally points to a gateway. The users table is independent and holds the user login and authentication details, role and approval status. With this normalized type of design, it's straightforward to query for the latest node read or to query for various time periods for charting.

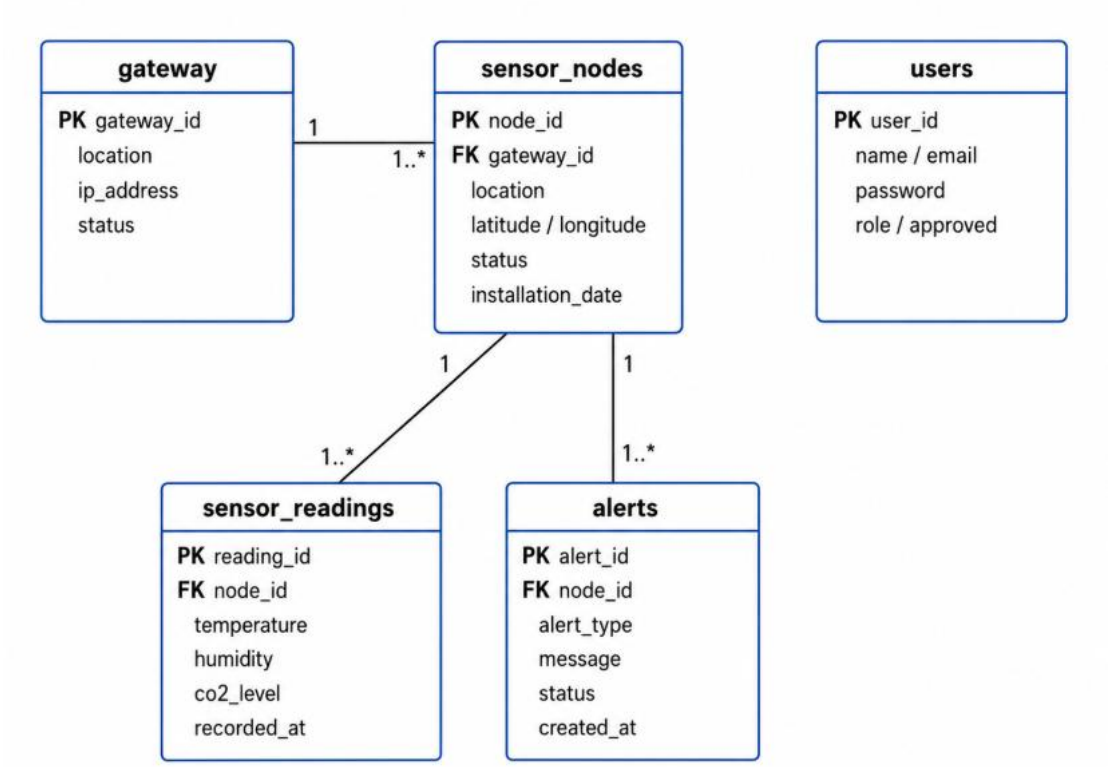


Figure 2.6. Database Schema (ERD) for the Forest Monitoring System

The schema was normalized to third normal form because there was no redundancy in the schema and to ensure that each reading or alert always refers to an existing node by using foreign-key constraints for referential integrity. Normally there is a one-to-one relationship between each node and a gateway, but this one relationship is here flattened to a 0:1 relationship, allowing for multi-gateway deployments where each zone of a large forest can have its own Raspberry Pi gateway, and the 1:0 prototype to continue to function in that way. The dashboard's live view and trend charts rely on an access pattern where the two most frequent queries that the system must solve are: "what is the latest reading for each node" and "what were the readings for this node over the past n hours.If we add an index to the node identifier and the timestamp to the sensor_readings table, we can enable these two frequent queries to be efficient even when the table grows in size.

4.7 Collaboration Diagram

The Collaboration diagram (Figure 4.7) highlights the collaborative efforts of the various components of the solution (the bridge Arduino, the gateway script, the database, the REST API and even the dashboard logic of alerts/receiving sounds) to display near real-time information on forest conditions continuously. It provides a complement to the sequence diagram and makes the structural connections between objects that collaborate more apparent than the temporal progression of the diagram, making it easier to see the relationships between the components.

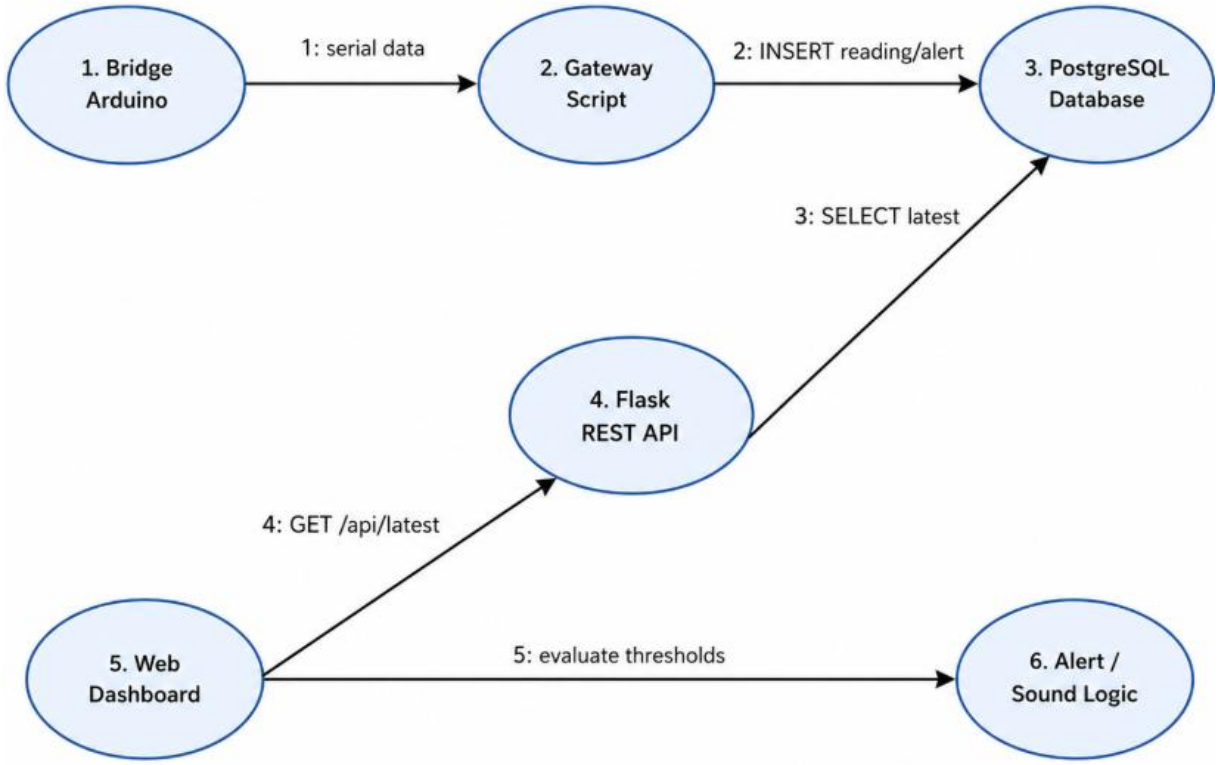


Figure 2.7. Collaboration Diagram for Gateway-to-Dashboard Data Flow

Chapter 5. Implementation

In this chapter, the hardware and software design directions are described. It contains work breakdown structure, project schedule, responsibilities of the functioning team, tools and technologies used, detailed implementation of each subsystem, screenshots and images of the working prototype and problems faced in the development.

5.1 Work Breakdown Structure (WBS)

The project was decomposed into seven principal tasks, each corresponding to one layer or concern of the system, as shown in Table 5.1. This decomposition mirrored the iterative lifecycle model and allowed each task to be developed and validated before the next was integrated.

Table5.1. Work Breakdown Structure (WBS)

Task	Description
1. Hardware Prototyping	Wire and test DHT11, MQ135, and 433 MHz RF modules on Node 1 and Node 2
2. RF Communication Protocol	Implement send/ACK/relay logic between Node 1, Node 2, and the bridge
3. Gateway Software	Develop gateway_insert.py for serial parsing and database insertion
4. Database Design	Design and create the PostgreSQL schema (gateway, sensor_nodes, sensor_readings, alerts, users)
5. Backend API	Develop the Flask REST API, including authentication and admin endpoints
6. Frontend Dashboard	Build the role-based web dashboard with live updates, charts, and alerts
7. Testing & Debugging	Unit, integration, and user-acceptance testing across the full pipeline

Table 5.2 presents the project schedule as a simplified phase timeline spanning the two FYP semesters, from initial planning through final testing and documentation.

Table5.2. Project Schedule (Sep 2025 – Jun 2026)

Phase	Duration	Key Deliverable
Requirement Analysis & Research	Sep – Oct 2025	Requirements & existing-solution study
Hardware Prototyping	Oct – Nov 2025	Working sensor nodes
Communication Protocol	Nov – Dec 2025	Reliable RF relay with ACK
Gateway & Database	Jan – Feb 2026	Ingestion pipeline & schema
Backend API	Feb – Mar 2026	Flask REST API

Phase	Duration	Key Deliverable
Frontend Dashboard	Mar – Apr 2026	Role-based dashboard
Testing & Documentation	Apr – Jun 2026	Validated system & thesis

5.2 Team Roles and Responsibilities

Responsibilities were divided so that each member owned a coherent vertical slice of the system while still collaborating closely at the integration boundaries, as summarized in Table 5.3.

Table 5.3. Team Roles and Responsibilities

Team Member	Primary Responsibility
Areeba Shahid	Hardware prototyping, RF communication protocol, gateway-script development
Nimra Faryal	Database design, backend REST API development
Afnan Fatima	Frontend dashboard development, role-based access design, testing

5.3 Tools and Technologies

Table 5.4 lists the hardware and software tools used across the project, grouped by category. The selection deliberately favored low-cost, widely available, open-source components in keeping with the project's core objective of affordability.

Table 5.4. Tools and Technologies

Category	Technology
Microcontrollers	Arduino Uno (Node 1, Node 2, Bridge)
Sensors	DHT11 (temperature/humidity), MQ135 (air quality / CO_2 estimation)
Wireless Communication	433 MHz ASK/OOK RF transmitter/receiver modules, RadioHead (RH_ASK) library
Gateway	Raspberry Pi 2 Model B
Gateway Software	Python 3, pyserial, psycopg2
Database	PostgreSQL 18 (pgAdmin 4)
Backend API	Flask, Flask-CORS
Frontend	HTML5, CSS3, JavaScript, Chart.js, Web Audio API
Development Tools	Arduino IDE, VS Code / Kiro, pgAdmin 4

5.4 Implementation Details

5.4.1 Hardware Implementation

Node 1/2 either had an Arduino Uno with an analog 433MHz transmitter and receiver pair with quarter wavelength wire soldered onto the antenna for greater range, and with a DHT11 temperature and humidity sensor (digital pin D7) and MQ135 air quality sensor (analog pin A0, with CO_2 ppm estimated by a calibrated resistance curve using load resistance R0 and curve-fit constants). To receive the broadcast from the node 2 and relay it to the Raspberry Pi 2 in plain serial text, only the bridge Arduino (with a 433 MHz receiver) was used and the dedicated bridge Arduino and the Raspberry Pi 2 connected using USB cable.

5.4.2 Communication Protocol

Node 2 should always be listening, and when it receives a packet from node 1 it should always reply with ACK and then it should read its own sensors including a combined packet and broadcast it. (eg. N1,...|N2,...) using the same radio module for both sends and receives. You can in fact have several listeners receive the same combined packet, since 433 MHz transmissions are broadcast nature – where Node 1 receives the packet to exchange the ACK and the bridge node receives the packet to store the combined data.

A plain-text, comma-and-pipe-delimited packet format was selected for a specific reason: At the low data rates and small packet sizes involved, the few extra bytes that are used are not significant in terms of transmission time, but they had a great benefit in the development. Each packet could be viewed in the serial monitor on the Arduino, and in the gateway log, which meant a bad reading or missing field was easy to spot with the naked eye, and not buried in unreadable bytes. The two nodes' sub-packets are separated from each other with the pipe character in one combined transmission, which helps the gateway parser know which node sent which reading with no positing required (other Nodes with the same sub-packet would be separated from this one by another pipe). Additionally, this format is easily extensible: adding a new node would involve adding another pipe delimited section, and the existing parser (which is split by the comma after splitting by the pipe), would easily handle it without having to be changed.

5.4.3 Gateway and Database

On the Raspberry Pi side, gateway_insert.py is used to read serial lines from the bridging Arduino, parsing out each individual node reading and inserting it into sensor_readings and, if it hits either a temperature threshold (38 °C / 45 °C) or a CO_2 threshold (1000 ppm), into the alerts table as well. The full PostgreSQL schema can be found in Appendix C.6, and the running of it in pgAdmin 4 is illustrated in Figure 6.1.

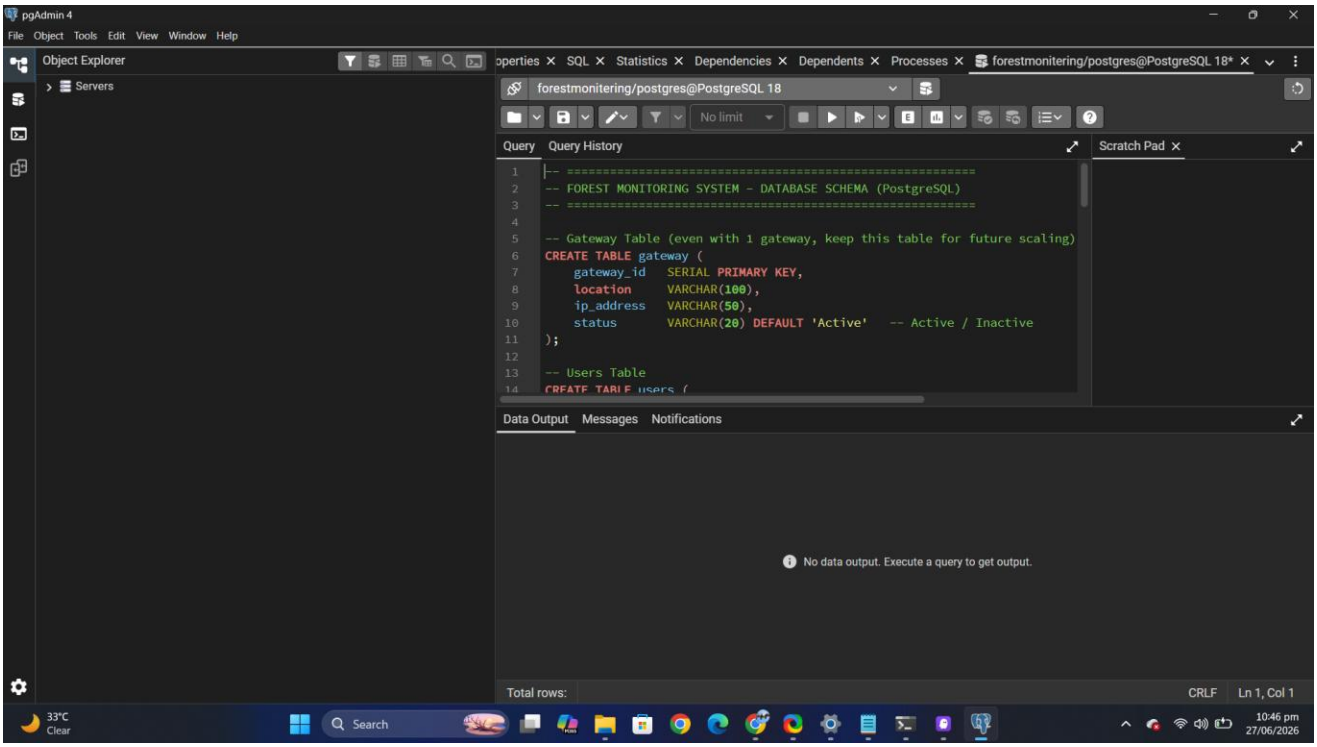


Figure 5.1. PostgreSQL Schema Execution in pgAdmin

.4.4 Backend API

Endpoints of a REST application are exposed: /api/latest (latest reading per node, gps coordinates are formatted as, for example, 33.7445 N, 72.7770 E), /api/history/<node_id> (recent readings for charting), /api/alerts, /api/alerts/full, /api/login, /api/register, /api/admin/table/<name> (inspection of raw tables by administrator).

5.4.5 Web Dashboard

They provide exact numeric values for all the different sensors, show flashing live value risk-labels, activate sound alerts (country-specific, via the Web Audio API), display the raw database tables and access user-management functionality, such as approving new user accounts, created through self-registration. The environmental-risk summary shown to the users is simplified with hazard indication via colors (Normal / Warning / Danger) only, and omitting details of raw numbers. Unlike maintaining a static database node status, the node Active/Inactive status is calculated client-side based on the last seen node reading, so the status of the dashboard always reflects the status of the node as to whether it is currently transmitting or not.

This was a subtle, but important feature design decision: Node liveness was computed on the client from the last-read time stamp in the database, not stored in the database as an explicit “active” flag. Some sort of process to actively clear a stored flag would have to exist when a node went silent, which leads to the classic stale flag problem which is exactly when a node failed and was no longer able to update its own flag. Relying on lack of recent data as an implicit signal of inactivity, the dashboard reports lack of data as an automatic node inactivity, again requiring no bookkeeping. The audible warning is also produced directly in the browser using the Web Audio API and not relying on an externally hosted soundfile to no longer introduce a network dependency, ensuring the warning gets sounded as soon as there is an imminent threat, even when the page is newly loaded.

5.4.6 Alert Threshold Configuration

The warning and danger levels and the type of warning to be issued triggered by a particular parameter's reaching a threshold level is defined in Table 5.5.

Table 5.5. Alert Threshold Configuration

Parameter	Warning Threshold	Danger Threshold	Alert Type
Temperature	≥ 38 °C	≥ 45 °C	High Temperature / Fire Risk
Humidity	≥ 85 %	N/A	Environmental Warning
CO_2 Level	≥ 700 ppm	≥ 1000 ppm	Smoke / Air Quality

These thresholds were selected as reasonable thresholds for a prototype demonstration and can be adjusted deployment-by-deployment by an Administrator in a future generation of the system based on the typical baseline conditions for each specific forest region.

5.5 Screenshots of Prototype / System

The following figures show the working prototype: the live dashboard with node readings (Figure 6.2), the role-based login screen (Figure 6.3), and the Administrator’s user-management panel (Figure 6.4).

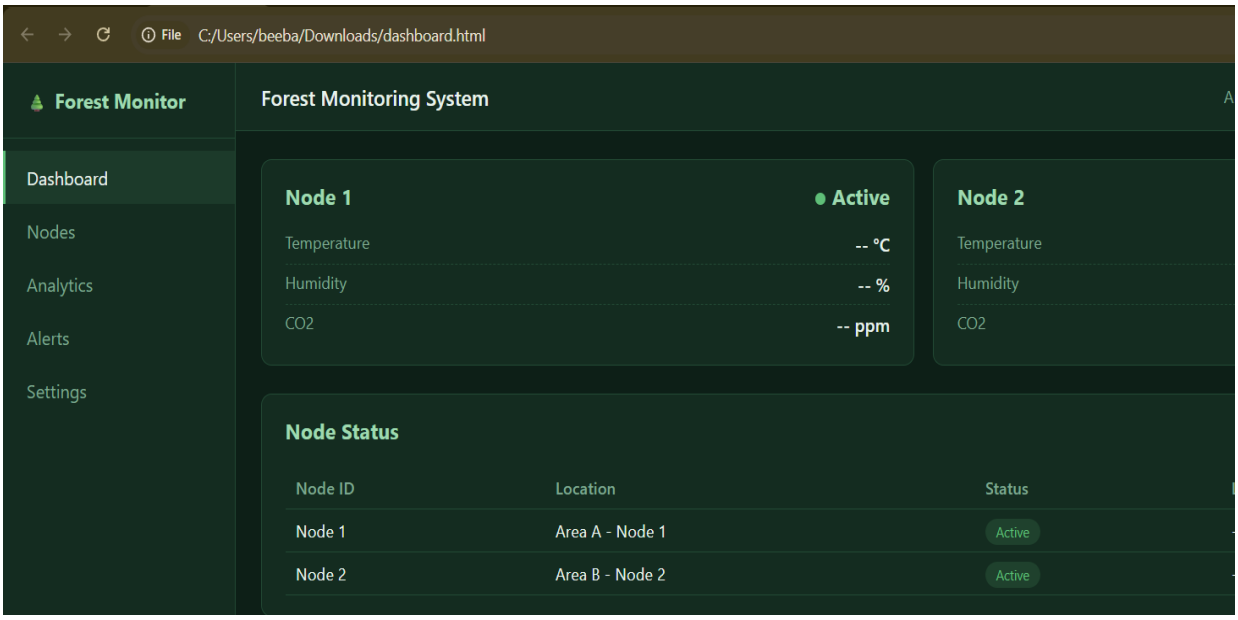


Figure 5.2. Forest Monitoring Dashboard — Live Node Readings

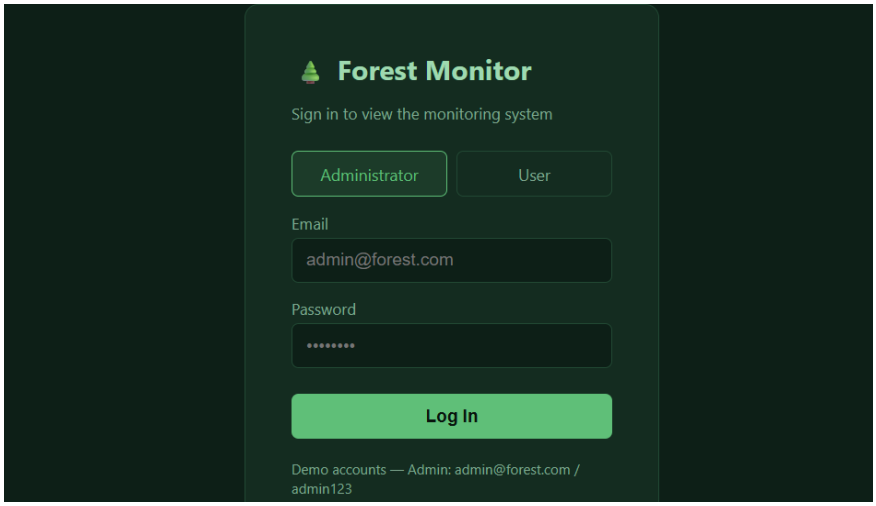


Figure 5.3. Role-Based Login Screen

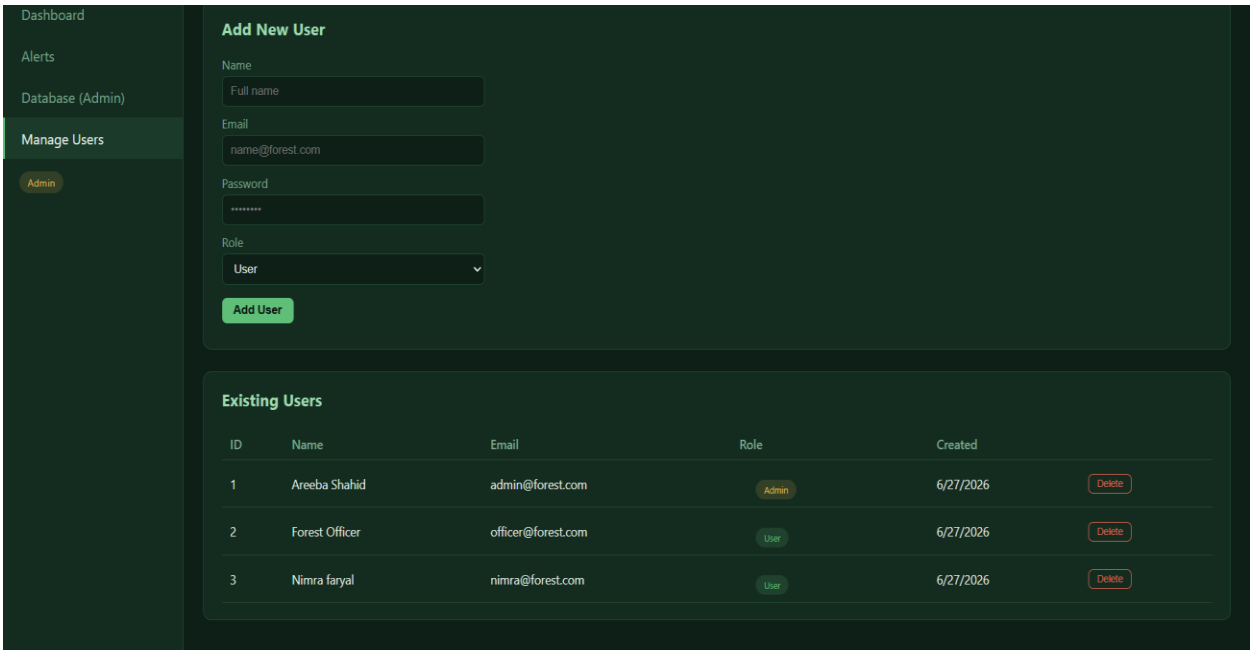


Figure 5.4. Administrator — Manage Users Panel

5.6 Challenges During Implementation

A number of practical issues arose during implementation, which did not surface during design, and which in a few cases were found to be a significant challenge that required considerable diagnostic effort to resolve. They are documented here, partly for completeness, but also because the way in which they are solved is more representative of practical aspects of designing a multi-technology embedded system. The major problems encountered were the following:

AVR floating-point formatting: the formatting for a float is the same on every node, but the formatting for human beings is not; when building the packet to transmit, one needs to perform the conversion with dtostrf, which is not built in as part of snprintf.

RH_ASK driver instability: By declaring two instances of the RH_ASK driver on the same Arduino, the system was continuously resetting; it was actually solved by broadcasting 433 MHz messages, since two instances can share the same transmitter/receiver driver, the driver for node 2 was consolidated.

Power-related resets: the MQ135 sensor caused brownout resets when powered solely though a laptop USB port in conjunction with the DHT11 and both RF modules on the same board - although these were infrequent.

Network/firewall configuration: Since the PostgreSQL installed on the Raspberry Pi needed to communicate with the database installed on the laptop, the Windows Firewall inbound rules were configured, as were the postgresql.conf and pg_hba.conf files to accept connections from other devices apart from localhost.

Database naming mismatch: early naming mismatch (forest monitoring versus forest monitoring) led to problem with connection failures (which were addressed by standardizing the name across all scripts)

There is a shared theme in these challenges every one occurred at the interstices between two technologies or platforms and required a common assumption to be made and none was “noisy” (none resulted in either system side reporting success and the other side reporting failure, or vice versa). This observation strengthened the testing strategy in effect since chapters 5, and guided the workflow of writing code from that point onward by explicitly logging every line parsed by the gateway and every address inserted by the API, to be able to determine the source of any future fault with ease rather than guesswork.

Chapter 6. System Testing

The testing was performed at various stages: testing of individual hardware modules and backend functions, testing of the entire pipeline, and user-acceptance testing of the two role based views. The goal was to verify the proper behavior under end-to-end both normal and threshold-exceeding conditions.

6.1 Test Cases Design

Test cases were created to include accuracy of the sensors, reliability of the RF-communication, integrity of the data in the database, correctness of the API, and the behavior of dashboard under both normal and "threshold exceeded" states. Every test case is required to define Description, Expected result and a degree of pass or fail as illustrated below in Table 6.1.

The testing strategy was to start with the most basic and verifiable component and work your way up to the entire system, since the sites consisted of five distinct technologies – firmware, ingestion script, database, web service and browser front end. This allowed for each piece of the end-to-end pipeline to be individually tested, before being integrated together so they could be run as a group, and to prove that they had to operate correctly when combined into the same context. Keeping up the separation of each interface from the hardware, the test engineer could then ensure that any fault found in integration testing could be blamed on the interface with confidence. Positive cases (where input was valid and output was correct) and negative cases (where input was invalid, acknowledgements were missing, or where issues with thresholds were raised, and the correct defensive stance was taken) were included in order to review the system not only to see if it is doing the right thing, but also if it is failing safely.

Table 6.1. Test Cases Design and Results (*range-dependent)

Test ID	Description	Expected Result	Status
TC-01	DHT11 reads temperature/humidity	Valid numeric values returned	Pass
TC-02	MQ135 returns calibrated CO_2 ppm	Non-NaN ppm within expected range	Pass
TC-03	Node 1 transmits packet, Node 2 receives it	Node 2 prints “Received from Node 1”	Pass
TC-04	Node 2 sends ACK, Node 1 receives it	Node 1 prints “ACK received”	Pass*
TC-05	Bridge relays combined packet via USB	Gateway prints “Inserted -> ...”	Pass
TC-06	API /api/latest returns correct JSON	JSON includes node_id, temperature, humidity, co2_level	Pass
TC-07	Login with valid Admin credentials	Redirect to Admin dashboard view	Pass

Test ID	Description	Expected Result	Status
TC-08	Login with unapproved account	“Pending approval” message shown	Pass
TC-09	Temperature ≥ 38 °C	Alert generated; banner and beep triggered	Pass
TC-10	Node inactive for > 15 seconds	Status switches from Active to Inactive	Pass

6.2 Unit Testing

Few test sketches were created for testing each hardware module separately so as to confirm that they received the correct sensor readings and raw packets were transmitted and received between the 433MHz transmitter and 433MHz receiver across a short range. The backend functions parse_single, insert_reading, and check_and_create_alert were tested by giving the gateway script some example serial lines and making sure the correct row (rows) were generated in the database (alert row as appropriate). In this case, each unit was isolated in such a way that when, later on, integration faults were manifested, they could be blamed on the interface between the components and not on the components themselves.

6.3 Integration Testing

The following problems had to get resolved during integration. Removed floating-point formatting issues on AVR GNU tools with Arduino – used dtostrf in place of %f when using snprintf. Driver instability RH_ASK was corrected by combining the Node 2 transmit receive driver, which was being defined multiple times, into a single instance. Power-Supply checks mitigated power-browndown caused by the MQ135 and causing board resets. Lastly, problems with PostgreSQL connections with the Windows Firewall blocking inbound connections and due to different naming used for the database (forestmonitering versus the assumed name of forestmonitoring) were addressed by configuring the Firewall and implementing consistent naming in all scripts.

Instead of installing all of the parts together, the integration testing took place gradually. We integrated and tested the RF connection between the 2 nodes, followed by the bridge-to-gateway serial handoff, then to the database insertion handoff, and finally through the REST API to the dashboard. The components were assembled sequentially, one interface at a time, resulting in the location of each fault to the one added interface. This discipline brought about the floating point, the driver-instability, and the brownout issues—all seemingly alike with the same symptom “no data to the dash”—separation from each other. For instance, in this case, you found that although the individual components pass their own tests, using them together it does not meet the desired requirements, such as reading content from the database, so this was a lesson in

a mismatch between the components at database-name level, and you can see that integration testing the interfaces is necessary even when you know each one is correct separately.

6.4 User Acceptance Testing

The Administrator and User roles have been tested manually to ensure that the dashboard is able to differentiate the access level. Administrators had access to precise numbers of sensor readings, the dynamic flashing threat notification banner, raw tables of data from all the databases, and the user-management features which included the acceptance of new users self-registered through the site. Users conversely only received a streamlined, colour coded, environmental-risk summary of the information and no raw numeric detail or administrative controls were provided. New-user registration, blocking pending approval, and follow-on approval by an Administrator, were all separately identified and tested against the specified security and usability characteristics with no deviations discovered.

User-acceptance testing also compared to the initial goal for the two-role design: a non-technical forest official should be able to look and understand the situation in the forest without having to interpret raw numbers. Colour-coded reviewers agreed that the color-coded summary was easily read – green means normal, amber means warning, and red means danger, and that the audible warning was effective at attracting attention if the dashboard wasn't the first focus of attention. All this was also verified via the Administrator view ensuring that nothing, not even direct navigation to administrative urls, would make all of the numeric detail and raw tables visible from a User-role session – there was no way for them to step up to privileged data through the simplified view. Combined, these tests validated the system meeting its functional requirements as well as usability and access-control desires which set it apart from a singular undifferentiated dashboard.

Chapter 7. Conclusions

7.1 Summary

This project successfully developed, implemented and evaluated an IoT-based Forest Monitoring System consisting of two wireless sensor nodes, 433 MHz multi-hop relay protocol, Raspberry Pi gateway, PostgreSQL database and flask REST API and role-based Web dashboard. The system is designed to constantly take data on temperature, humidity, and CO_2; reliably share the data between nodes (with a request-acknowledge mode); and save the data in the centralized way and display the data on dashboard that differentiate two authority: Administrator and User. A flashing visual banner and audible alarm were added to assist with fire-risk detection early enough, directly addressing the problem found at the start of this project relating to fire-risk detection being later than intended by manual checks. The system achieves its original goals of affordability, modularity, and extensibility, and underwent unit testing as well as integration and user-acceptance testing.

In addition to achieving its original objectives, the project provides an example for others: An end-to-end environmental-monitoring system can be built from commercial off-the-shelf hardware and open-source software without ongoing subscription fees or proprietary infrastructure. The approach is specifically applicable to deployments in resource-limited and developing regions, where many of the commercial options discussed in Chapter 2 are hard to implement.

During the development process additional lessons were also obtained, which could be extended to further developments than the artefact produced. The most interesting of these was the realisation that, in an ultra-tight hardware/software system, the interfaces between the hardware and software components were the most dangerous: The most infuriating failures we found were failures of floating-point formatting in the AVR toolchain, failures in instabilities in the radio driver, failures due to sensor-induced brownout, and failures caused by not matching the name of the database used. Layered architectures and bottom up test approach were exactly the just thing that made these mismatches manageable. A second lesson was that designing with the end user in mind - a non-technical forest official - yields material results: thinking through how to use the same underlying set of data in a summary of safety at a glance and a console for administrators came together as the two-role design.

7.2 Recommendations for Future Work

The prototype demonstrates the entire sensing-to-dashboard "pipeline," but there are a few improvements that would need to be made to transition from a working system to sustainable field deployment. These can be categorized as three types: Expanding Physical Coverage and Autonomy, Expanding Alert Reach and Expanding Analysis Intelligence. The following directions are the most important recommended:

- Adding more nodes with the sensors to cover more spatial area in the forest.
- Solar-Powered and battery managed to support completely autonomous and long-term outdoor deployments.
- Use of a lower power radio that can transmit over a greater distance (e.g. LoRa).
- Incorporating SMS or email notification integration for remote notifications beyond the Web application dashboard.
- On a server located on a public cloud with a real domain, so that it can be accessed from anywhere and is possible to enter the "real" third-party authentication (e.g. Google Sign-In).
- Weather proof enclosure and ruggedized wiring for long term field deployment.
- Importing prediction based on machine learning, considering the short term trends of environmental factors instead of only thresholds.

Chapter 8. References

[1] G. Brundtland, Report of the World Commission on Environment and Development: Our Common Future, United Nations, 1987.

[2] A. Marchi, L. Chung, and A. Visser, “Sustainable Forest Operations (SFO): A Comprehensive Approach to Forest Management,” Forests, vol. 9, no. 10, pp. 1–14, 2018.

[3] H. Kopetz, “Internet of Things — From Science Fiction to Reality,” IEEE IoT Journal, vol. 1, no. 1, pp. 3–9, 2014.

[4] R. Singh, A. Gehlot, S. Vaseem, and A. Akram, “Digitalization of Forest Using Internet of Things (IoT): A Comprehensive Review,” Journal of King Saud University — Computer and Information Sciences, vol. 34, no. 7, pp. 5587–5601, 2022.

[5] Food and Agriculture Organization of the United Nations (FAO), Global Forest Resources Assessment, Rome, Italy, 2020.

[6] Intergovernmental Panel on Climate Change (IPCC), Climate Change 2021: The Physical Science Basis, Geneva, Switzerland, 2021.

[7] J. A. Foley et al., “Global Consequences of Land Use,” Science, vol. 309, no. 5734, pp. 570–574, 2005.

[8] B. K. Sovacool, “The Increasing Threat of Wildfires in a Changing Climate,” Environmental Research Letters, vol. 13, no. 3, pp. 1–7, 2019.

[9] WWF, Living Planet Report: Forest Biodiversity and Wildlife Crisis, World Wildlife Fund, 2022.

[10] P. Das, R. Roy, and M. Islam, “An IoT-Based Real-Time Forest Environmental Monitoring System,” Journal of Ambient Intelligence and Humanized Computing, vol. 13, no. 2, pp. 913–923, 2022.

[11] M. Yadav, S. Rao, and P. Jain, “Design and Implementation of a Smart Environmental Monitoring System Using IoT,” International Journal of Engineering Science and Technology, vol. 29, no. 5, pp. 455–462, 2022.

[12] L. Zhou and Y. Zhang, “Satellite-Based Remote Sensing for Forest Fire Surveillance and Detection,” IEEE Geoscience and Remote Sensing Letters, vol. 18, no. 9, pp. 1565–1570, 2021.

[13] H. Rahman and S. Ullah, “IoT-Based Early Wildfire Detection and Monitoring System,” in Proc. Int. Conf. Sustainable Computing and Systems, Dhaka, Bangladesh, 2022, pp. 1–6.

[14] J. Patel, M. Mehta, and R. Thakkar, “Smart Forest Fire Detection and Environmental Monitoring using LoRa,” International Journal of Innovative Research in Science, Engineering and Technology, vol. 10, no. 3, pp. 884–893, 2022.

[15] H. Al-Janabi, M. Al-Khazraji, and I. Hamad, “Wireless Sensor-Based Remote Environmental Monitoring for Forests,” Journal of Engineering Science and Technology, vol. 16, no. 1, pp. 83–96, 2021.

[16] R. Singh, A. Kumar, and V. Sharma, “IoT-Based Wireless Sensor Network for Real-Time Forest Monitoring and Fire Detection,” IEEE Access, vol. 11, pp. 20832–20845, 2023.

[17] RadioHeadLibrary Documentation, Mike McCauley. https://www.airspayce.com/mikem/arduino/RadioHead/

[18] Adafruit DHT Sensor Library, Adafruit Industries. https://github.com/adafruit/DHT-sensor-library

[19] Flask Documentation, Pallets Projects. https://flask.palletsprojects.com/

[20] PostgreSQL Documentation, The PostgreSQL Global Development Group. https://www.postgresql.org/docs/

[21] RaspberryPi Documentation, Raspberry Pi Foundation. https://www.raspberrypi.com/documentation

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- **88** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **54** Missing Citation 5%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
3% Publications
13% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.